

**LAPORAN TUGAS AKHIR PRAKTIKUM 2 -
STRUKTUR SISTEM OPERASI**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Nabila Senja Ramadhani (2410651043)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas Akhir Praktikum

Tugas 1: Sistem Manajemen Proses

Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur:

Membuat multiple child processes

Setiap process melakukan tugas berbeda (sorting, searching, calculation)

Parent process mengkoordinasi dan mengumpulkan hasil

Tampilkan PID, execution time, dan status setiap proses

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano process_manager.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc -Wall -O2 -o process_manager process_manager.c
senjaramadhani@LAPTOP-823QCJCF:~$ ./process_manager
~bash: ./process_manager: No such file or directory
senjaramadhani@LAPTOP-823QCJCF:~$ ./process_manager
Parent PID: 572. Waiting for children...

Child 0:
PID : 573
Exec time : 0.002213 sec
Status : Exited (code 0)
Result : first5: 96 147 152 260 334 | last5: 999523 999774 999883 999849 999881

Child 1:
PID : 574
Exec time : 0.000707 sec
Status : Exited (code 0)
Result : target 142819 NOT FOUND (closest index 71410)

Child 2:
PID : 575
Exec time : 0.000026 sec
Status : Exited (code 0)
Result : Fib(47) = 2971215873

All children finished.
senjaramadhani@LAPTOP-823QCJCF:~$ |
```

Hasil akhirnya adalah berhasil saya compile dan jalankan

Hasil akhir source code lengkap setelah melalui error:

```
/ process_manager.c
```

```
#define _GNU_SOURCE
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <sys/types.h>
```

```
#include <time.h>
```

```
#include <string.h>
```

```
#include <errno.h>
```

```
#define NUM_CHILDREN 3
```

```
#define RESULT_SIZE 256
```

```
typedef struct {    pid_t pid;    double exec_time;  
    // seconds    int status_code;    // 0 = success, non-  
    zero = error    char result[RESULT_SIZE];  
} msg_t;
```

```
/* ===== Utility: hitung selisih waktu dalam detik ===== */ static double  
timespec_diff_sec(const struct timespec *start, const struct timespec  
*end) {    double s = (double)(end->tv_sec - start->tv_sec);  
    double ns = (double)(end->tv_nsec - start->tv_nsec) / 1e9;  
    return s + ns;  
}
```

```
/* ===== Comparator untuk qsort ===== */  
int int_cmp(const void *a, const void *b) {  
    int ai = *(const int*)a;    int bi = *(const  
    int*)b;    return (ai > bi) - (ai < bi);  
}
```

```
/* ===== Task 1: Sorting ===== */ void  
task_sorting_ok(char *out, size_t outsz) {    const int N  
= 20000;    int *arr = malloc(sizeof(int) * N);    if (!arr)  
{ snprintf(out, outsz, "malloc fail"); return; }
```

```
    for (int i = 0; i < N; ++i) arr[i] = rand() % 1000000;
```

```
    qsort(arr, N, sizeof(int), int_cmp);
```

```

    // Ringkasan hasil char
    tmp[RESULT_SIZE];

    int p = 0;    p += snprintf(tmp + p, sizeof(tmp) - p, "first5: ");
    for (int i = 0; i < 5 && i < N; ++i)    p += snprintf(tmp + p,
    sizeof(tmp) - p, "%d ", arr[i]);    p += snprintf(tmp + p, sizeof(tmp)
    - p, "| last5: ");    for (int i = N - 5; i < N; ++i)    if (i >= 0) p +=
    snprintf(tmp + p, sizeof(tmp) - p, "%d ", arr[i]);

    strncpy(out, tmp, outsz - 1);
    out[outsz - 1] = '\0';
    free(arr);
}

/* ===== Task 2: Searching ===== */ void
task_searching(char *out, size_t outsz) {
    const int N = 100000;    int *arr =
    malloc(sizeof(int) * N);

    if (!arr) { snprintf(out, outsz, "malloc fail"); return; }

    for (int i = 0; i < N; ++i) arr[i] = i * 2; // array sorted
    int target = rand() % (N * 2);

    int l = 0, r = N - 1, found = -1;
    while (l <= r) {    int m = l +
    (r - l) / 2;

    if (arr[m] == target) { found = m; break; }
    else if (arr[m] < target) l = m + 1;

    else r = m - 1;
}

```

```

        if (found >= 0)        snprintf(out, outsz, "target %d FOUND at index %d",
target, found);    else        snprintf(out, outsz, "target %d NOT FOUND (closest
index %d)", target, l);

```

```

        free(arr);
    }

```

```

/* ===== Task 3: Calculation ===== */ void
task_calculation(char *out, size_t outsz) {
    const int N = 45 + (rand() % 6);    unsigned
    long a = 0, b = 1;    for (int i = 2; i <= N; ++i)
    {        unsigned long c = a + b;        a = b;
    b = c;
    }
    snprintf(out, outsz, "Fib(%d) = %lu", N, b);
}

```

```

/* ===== Jalankan tugas anak ===== */ int
run_child_task(int child_id, msg_t *m) {
    char result[RESULT_SIZE] = {0};    struct
    timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    srand(((unsigned int)(time(NULL) ^ getpid() ^ (child_id * 123456))));

    if (child_id == 0) task_sorting_ok(result, sizeof(result));    else if
    (child_id == 1) task_searching(result, sizeof(result));    else if
    (child_id == 2) task_calculation(result, sizeof(result));    else
    snprintf(result, sizeof(result), "unknown task id %d", child_id);

    clock_gettime(CLOCK_MONOTONIC, &t1);

```

```

    m->pid = getpid();    m->exec_time =
timespec_diff_sec(&t0, &t1);    m->status_code
= 0;    strncpy(m->result, result, RESULT_SIZE
- 1);    m->result[RESULT_SIZE - 1] = '\0';
return 0;
}

```

```

/* ===== Main Program ===== */ int
main(void)    {                int
pipes[NUM_CHILDREN][2];
pid_t pids[NUM_CHILDREN];

```

```

    for (int i = 0; i < NUM_CHILDREN; ++i) {
if (pipe(pipes[i]) == -1) {        perror("pipe");
return 1;
    }
}

```

```

    for (int i = 0; i < NUM_CHILDREN; ++i) {
pid_t pid = fork();                if (pid < 0) {
perror("fork");
    return 1;
    } else if (pid == 0) {        for (int j = 0; j <
NUM_CHILDREN; ++j) {                if (j != i) {
close(pipes[j][0]);                close(pipes[j][1]);
    }
    }
close(pipes[i][0]);

```

```

        msg_t m;
run_child_task(i, &m);

        // tulis hasil ke parent dan periksa return value        ssize_t w =
write(pipes[i][1], &m, sizeof(m));        if (w != (ssize_t)sizeof(m)) {
fprintf(stderr, "Child %d (PID %d): write() failed, wrote %zd bytes\n",
i, getpid(), w);
        }

        close(pipes[i][1]);
        _exit(m.status_code);
    } else {
pids[i] = pid;
close(pipes[i][1]);
    }
}

printf("Parent PID: %d. Waiting for children...\n\n", getpid());

for (int i = 0; i < NUM_CHILDREN; ++i) {
    int status;        pid_t w = waitpid(pids[i],
&status, 0);        msg_t m;        ssize_t r =
read(pipes[i][0],        &m,        sizeof(m));
close(pipes[i][0]);

    if (r <= 0) {        snprintf(m.result, sizeof(m.result), "No
result (read=%zd)", r);        m.pid = w;
        m.exec_time = 0.0;
        m.status_code = -1;
    }
}

```

```

        char status_str[64];
        if (WIFEXITED(status))
            snprintf(status_str, sizeof(status_str), "Exited (code %d)",
WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            snprintf(status_str, sizeof(status_str), "Killed (signal %d)",
WTERMSIG(status));
        else
            snprintf(status_str, sizeof(status_str), "Unknown");

        printf("Child %d:\n", i);    printf(" PID      :
%d\n", m.pid);    printf(" Exec time  : %.6f sec\n",
m.exec_time);    printf(" Status      : %s\n",
status_str);    printf(" Result     : %s\n\n", m.result);
    }

    printf("All children finished.\n");
    return 0;
}

```


Tugas 2: Chat Application dengan IPC

Implementasikan aplikasi chat sederhana menggunakan:

Shared Memory atau Message Queue untuk menyimpan pesan

Multiple processes sebagai pengguna

Mutex/Semaphore untuk sinkronisasi

Fitur: kirim pesan, lihat history, exit

```
senjaramadhani@LAPTOP-823QCJCF:~$ nano chat.c
senjaramadhani@LAPTOP-823QCJCF:~$ gcc -Wall -O2 -o chat chat.c -lrt -pthread
senjaramadhani@LAPTOP-823QCJCF:~$ ./chat
=== Simple IPC Chat ===
Masukkan username (maks 31 chars): Nabilasenja

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 1
Tulis pesan (max 255 chars). Tekan Enter:
> Halo Semua!
Terkirim.

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 1
Tulis pesan (max 255 chars). Tekan Enter:
> Halo User!!
Terkirim.

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 1
Tulis pesan (max 255 chars). Tekan Enter:
> Bagaimana Kabarmu?
Terkirim.

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 1
Tulis pesan (max 255 chars). Tekan Enter:
> Baik, terima kasih
Terkirim.

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 1
Tulis pesan (max 255 chars). Tekan Enter:
> Bagaimana Kabarmu?
Terkirim.

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 1
Tulis pesan (max 255 chars). Tekan Enter:
> Baik, terima kasih
Terkirim.

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 2
=== History (4 pesan) ===
[2025-10-24 06:49:29] Nabilasenja (pid:1038): Halo Semua!
[2025-10-24 06:49:37] Nabilasenja (pid:1038): Halo User!!
[2025-10-24 06:49:55] Nabilasenja (pid:1038): Bagaimana Kabarmu?
[2025-10-24 06:50:06] Nabilasenja (pid:1038): Baik, terima kasih
=== End ===

[Nabilasenja@pid:1038] Menu:
1) Kirim pesan
2) Lihat history
3) Exit
Pilih (1/2/3): 3
Goodbye.
senjaramadhani@LAPTOP-823QCJCF:~$ |
```

Di akhir ini bisa kita liat code yang saya compile dan jalankan ialah berhasil, disini bisa kita lihat kita dapat menjalankan 2 perintah akhir atau (run) seperti tertera di foto

Source code lengkap:

/* chat.c

Simple multi-process chat using POSIX shared memory + named semaphore

Compile: gcc -Wall -O2 -o chat chat.c -lrt -pthread

Usage:

./chat -> run one client (open multiple terminals to simulate multiple users)

./chat -f N -> fork N child clients in same terminal (for testing) */

```
#define _POSIX_C_SOURCE 200809L
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h> // O_* constants
```

```
#include <sys/mman.h>
```

```
#include <sys/stat.h> // mode constants
```

```
#include <semaphore.h>
```

```
#include <time.h>
```

```
#include <errno.h>
```

```
#include <signal.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#define SHM_NAME "/chat_shm_example_2025"
```

```
#define SEM_NAME "/chat_sem_example_2025"
```

```
#define MAX_USERLEN 32
```

```
#define MAX_TEXT 256
```

```
#define MAX_MESSAGES 100
```

```
#define MAX_USERBUF (MAX_USERLEN * 2)
```

```

typedef struct {
    time_t      t;
    pid_t pid;
    char user[MAX_USERLEN];
    char text[MAX_TEXT];
} message_t;

```

```

typedef struct {
    int initialized;
    int refcount;
    int head;    int
count;
    message_t msgs[MAX_MESSAGES];
} chat_shm_t;

```

```

static chat_shm_t *chat = NULL; static sem_t
*mutex = NULL; static int shm_fd = -1; static
char username[MAX_USERLEN] = {0};

```

```

void cleanup_and_exit(int code);

```

```

void perror_exit(const char *msg) {
    perror(msg);
    cleanup_and_exit(EXIT_FAILURE);
}

```

```

void attach_shared_resources() {    mutex =
sem_open(SEM_NAME, O_CREAT, 0666, 1);    if
(mutex == SEM_FAILED) {
    perror_exit("sem_open");
}
}

```

```

    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {        perror_exit("shm_open");
    }

    size_t shm_size = sizeof(chat_shm_t);
    if (ftruncate(shm_fd, shm_size) == -1) {
    perror_exit("ftruncate");
    }

    void *addr = mmap(NULL, shm_size, PROT_READ | PROT_WRITE,
    MAP_SHARED, shm_fd, 0);    if (addr == MAP_FAILED) {
    perror_exit("mmap");
    }
    chat = (chat_shm_t *)addr;

    if (sem_wait(mutex) == -1) perror_exit("sem_wait");
    if (!chat->initialized) {    chat->initialized = 1;
    chat->refcount = 0;        chat->head = 0;        chat-
    >count = 0;        for (int i = 0; i < MAX_MESSAGES;
    ++i) {                chat->msgs[i].t = 0;        chat-
    >msgs[i].pid = 0;        chat->msgs[i].user[0] = '\0';
    chat->msgs[i].text[0] = '\0';
    }
    }
    chat->refcount += 1;
    if (sem_post(mutex) == -1) perror_exit("sem_post");
}

void detach_shared_resources() {    if
(!chat) return;        size_t shm_size =

```

```
sizeof(chat_shm_t);    munmap(chat,  
shm_size);    chat = NULL;
```

```
    if (shm_fd != -1) {  
        close(shm_fd);  
shm_fd = -1;  
    }
```

```
    if (mutex != NULL && mutex != SEM_FAILED) {  
sem_close(mutex);    mutex = NULL;  
    }  
}
```

```
void cleanup_and_exit(int code) {    if  
(mutex != NULL && chat != NULL) {  
if (sem_wait(mutex) == 0) {        if  
(chat->refcount > 0)            chat-  
>refcount -= 1;            int ref = chat-  
>refcount;            sem_post(mutex);  
if (ref == 0) {  
                shm_unlink(SHM_NAME);  
sem_unlink(SEM_NAME);  
            }  
        }  
    }  
    detach_shared_resources();  
exit(code);  
}
```

```
void sigint_handler(int signo) {
```

```

        (void)signo;
printf("\nExiting...\n");
        cleanup_and_exit(EXIT_SUCCESS);
}

```

```

void print_message(const message_t *m) {    char timestr[64];    struct tm
tm;    localtime_r(&m->t, &tm);    strftime(timestr, sizeof(timestr), "%Y-
%m-%d %H:%M:%S", &tm);    printf("[%s] %s (pid:%d): %s\n", timestr,
m->user, (int)m->pid, m->text);
}

```

```

void show_history() {    if
(sem_wait(mutex) == -1) {
perror("sem_wait");
return;
}
    int c = chat->count;    int idx
= chat->head;    if (c == 0) {
printf("History kosong.\n");
    } else {
        printf("=== History (%d pesan) ===\n", c);
for (int i = 0; i < c; ++i) {
            int cur = (idx + i) % MAX_MESSAGES;            print_message(&chat-
>msgs[cur]);
        }
        printf("=== End ===\n");
    }
    if (sem_post(mutex) == -1) perror("sem_post");
}

```

```

void send_message(const char *text) {

```

```

    message_t m;
    m.t = time(NULL);
    m.pid = getpid();    strncpy(m.user, username,
MAX_USERLEN - 1);    m.user[MAX_USERLEN
- 1] = '\0';    strncpy(m.text, text, MAX_TEXT - 1);
m.text[MAX_TEXT - 1] = '\0';

    if (sem_wait(mutex) == -1) {
perror("sem_wait");
return;
    }

    int tail = (chat->head + chat->count) % MAX_MESSAGES;
chat->msgs[tail] = m;    if (chat->count < MAX_MESSAGES)
{
    chat->count += 1;
    } else {
    chat->head = (chat->head + 1) % MAX_MESSAGES;
    }

    if (sem_post(mutex) == -1) perror("sem_post");
}

void run_client_interactive() {
char choice[8];    char
buf[MAX_TEXT];
    while (1) {        printf("\n[%s@pid:%d] Menu:\n", username,
(int)getpid());        printf(" 1) Kirim pesan\n");        printf(" 2)
Lihat history\n");

        printf(" 3) Exit\n");        printf("Pilih
(1/2/3): ");        fflush(stdout);        if

```

```

(!fgets(choice, sizeof(choice), stdin)) {
printf("\nEOF detected, exiting.\n");
break;
    }
    int opt = atoi(choice);    if (opt == 1) {        printf("Tulis pesan
(max %d chars). Tekan Enter:\n> ", MAX_TEXT - 1);        fflush(stdout);
if (!fgets(buf, sizeof(buf), stdin)) continue;        size_t L = strlen(buf);
if (L && buf[L - 1] == '\n') buf[L - 1] = '\0';        if (strlen(buf) == 0) continue;
send_message(buf);        printf("Ter kirim.\n");    } else if (opt == 2) {
show_history();    } else if (opt == 3) {        printf("Goodbye.\n");
break;    } else {        printf("Pilihan tidak valid.\n");
    }
}
}

```

```

int main(int argc, char *argv[]) {
signal(SIGINT, sigint_handler);
signal(SIGTERM, sigint_handler);
int fork_mode = 0;    int fork_N = 0;
if (argc >= 3 && strcmp(argv[1], "-
f") == 0) {        fork_mode = 1;
fork_N = atoi(argv[2]);    if (fork_N
< 1) fork_N = 1;
    }
}

```

```

printf("=== Simple IPC Chat ===\n");

```

```

printf("Masukkan username (maks %d chars): ", MAX_USERLEN - 1);
fflush(stdout);    if (!fgets(username, sizeof(username), stdin)) {
fprintf(stderr, "Gagal membaca username.\n");    return 1;
}

```



```

    size_t L = strlen(username);    if (L && username[L -
1] == '\n') username[L - 1] = '\0';    if (username[0] ==
'\0') {
        snprintf(username, sizeof(username), "user%d", (int)getpid());
printf("Username kosong -> memakai %s\n", username);
    }

    attach_shared_resources();

    if (fork_mode) {        for (int i =
1; i < fork_N; ++i) {            pid_t
pid = fork();            if (pid < 0) {
perror("fork");

                break;
            } else if (pid == 0) {                char
buf[MAX_USERBUF];                snprintf(buf, sizeof(buf),
"%s_child%d", username, i);                strncpy(username, buf,
sizeof(username) - 1);                username[sizeof(username) -
1] = '\0';                run_client_interactive();
cleanup_and_exit(EXIT_SUCCESS);
            }
        }

        run_client_interactive();        while
(wait(NULL) > 0) {}
cleanup_and_exit(EXIT_SUCCESS);
    } else {        run_client_interactive();
cleanup_and_exit(EXIT_SUCCESS);
    }

    return 0;
}

```